Elo-MMR: replicating and extending a rating system for massive competitions

Kanhav Purohit (240150014) · Arnav Sinha (240150005)

Replication of Ebtekar & Liu, The Web Conference 2021 (arXiv:2101.00400), tested on Codeforces, synthetic data and a new AtCoder dataset

Introduction → Proposed idea → Architecture → Results → Conclusion · then code demo and Q&A

INTRODUCTION

Rating thousands of players from rankings alone is hard

- Programming contests, races and judged sports only produce a ranking per event; raw scores can't be compared across events
- Elo and Glicko are built for 1-vs-1 games; TrueSkill handles many players but has no proven guarantees
- Codeforces and TopCoder use ad-hoc formulas, and some can be gamed by losing on purpose
- Our task: reimplement Elo-MMR, test whether its results reproduce, explain any gaps, and extend it

System	Massive	Monotonic	Robust
Glicko-2 / TopCoder	—	no	**yes**
Codeforces	no	**yes**	no
TrueSkill	unproven	unproven	no
Elo-MMR	**proven**	**proven**	**proven**

Massive = linear time · Monotonic = it never pays to lose · Robust = one round can't wreck a rating

PROPOSED IDEA

A Bayesian model where only the ranking is observed



Joint model (Eq. 1): prior × skill drift × performance noise × ranking

$$\Pr(S, P, E) = \prod_i \Pr(S_{i,0}) \prod_{i,t} \Pr(S_{i,t} \mid S_{i,t-1}) \prod_{i,t} \Pr(P_{i,t} \mid S_{i,t}) \prod_t \Pr(E_t \mid P_{\cdot,t})$$

- Each player's performance is their skill plus noise (Gaussian, or heavy-tailed logistic); only the ranking is seen
- Key insight: with thousands of players, a ranking pins down each performance almost exactly (Doob's consistency theorem)
- So players decouple: the update is two 1-D problems per player, first find the performance, then the new rating

Rating = most probable skill (MAP)

$$\mu_{i,t} = \arg\max_s \pi_{i,t}(s) \Pr(P_{i,t} = p_{i,t} \mid S_{i,t} = s)$$

prior × likelihood of this round's performance

Each round: two one-dimensional solves per player

Phase 1 · performance from the ranking

$$Q_i(p) = \sum_{j \succ i} l_j(p) + \sum_{j \sim i} d_j(p) + \sum_{j \prec i} v_j(p) = 0$$

$$F_j(p) = \frac{1}{1 + e^{-(p - \mu_j)/\bar{\delta}_j}}, \quad \delta_j^2 = \sigma_j^2 + \beta^2, \quad \bar{\delta}_j = \frac{\sqrt{3}}{\pi}\,\delta_j$$

$$l_j(p) = -\frac{F_j(p)}{\bar{\delta}_j}, \qquad v_j(p) = \frac{1 - F_j(p)}{\bar{\delta}_j}, \qquad d_j = l_j + v_j$$

Losses pull p down, wins push it up; p is where expected rank equals actual rank. Q is decreasing, so Newton finds the unique root.

Phase 2 · rating from all performances

$$L'(s) = \frac{s - p_0}{\beta_0^2} + \sum_k \frac{\pi}{\beta_k \sqrt{3}} \tanh\left(\pi\,(s - p_k) \,/\, \beta_k \sqrt{12}\right) = 0$$

$$\mu_t = \frac{\sum_k w_k\, p_k}{\sum_k w_k}, \qquad \frac{1}{\sigma_t^2} = \sum_k \frac{1}{\beta_k^2}$$

The rating is a weighted average of past performances. Each weight is about 1/β² for a typical round but shrinks toward 0 for a freak round, so outliers barely move the rating (the “R” in MMR).

Between rounds: skill drift, and our extension

Pseudodiffusion (paper, Algorithm 2)

$$\kappa = (1 + \gamma^2/\sigma^2)^{-1}$$

$$w_k \leftarrow \kappa^{1+\rho} w_k, \qquad w_0 \leftarrow \kappa w_0 + (\kappa - \kappa^{1+\rho}) \sum_k w_k$$

Old rounds fade; their weight moves onto a Gaussian centred on the current rating, so uncertainty grows by γ^2 while the rating stays put.

Our extension: time-aware drift

$$\gamma^2_{i,t} = w\,\sigma^2_{\text{lim}} + c\,\Delta\text{days}_{i,t}$$

$$\beta^2 = (1 + 1/w)\,\sigma^2_{\text{lim}}$$

The paper adds drift per round played only. We add c per day away (c = 3–10 rating²/day). It never reads performances, so "never pays to lose" still holds.

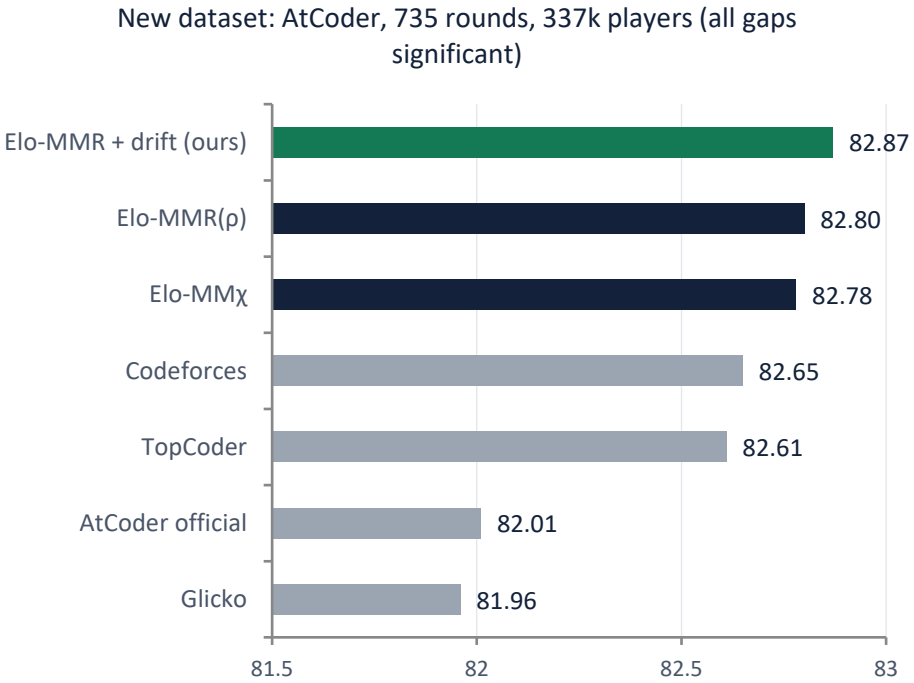
Robust (Thm 5.7)

$$|\mu_t - \mu_{t-1}| \le \frac{\pi\,\beta_0^2}{\beta_t\sqrt{3}}$$

Implementation: Numba-parallel kernels; opponents grouped by (μ, σ); nearest 500 groups per player; 32 stored rounds per player. Evaluation: tune on the first 10% of rounds, score the last 90%, players with ≥ 5 past rounds.

RESULTS

The paper reproduces, and its ranking holds on new data



Replication on the paper's data

Claim	Paper	Ours
Codeforces accuracy	78.6	**78.61**
Synthetic accuracy	81.7	**81.63**
Speed vs exact Codeforces	6×	**13×**

Our time drift helps in every setting
+0.02 to +0.10 points; every 95% interval is above zero on both Codeforces and AtCoder (bootstrap over rounds).

Caveat: Codeforces figures use the authors' settings; tuned on the first 10%, TopCoder edges Elo-MMR(ρ), 78.62 vs 78.37.

Conclusions and limitations

CONCLUSIONS

- Elo-MMR's accuracy, synthetic results and speed reproduce once our setup matches the paper's
- Every first-pass gap had a testable cause: tuning split, participation rate, or how the baseline was coded
- On AtCoder, data the paper never saw, Elo-MMR beats every baseline and the platform's own ratings, all with 95% confidence
- Time-aware drift is a small, consistent, incentive-safe gain

LIMITATIONS

- Sensitive to tuning: with the paper's 10% split, TopCoder beats tuned Elo-MMR(ρ) on Codeforces
- Our gains are modest (+0.02 to +0.10 points); adaptive round weighting did not help
- Nearest-500 subsampling breaks exact monotonicity (up to 0.3 points in our test)
- No TopCoder, Reddit or TrueSkill comparison: that data was never published
- Metrics only score ordering; whether σ is calibrated is untested

Next: code demo · Q&A